FlakeXplain

Completed study and independent confirmation - 14 October 2026

The result to present

We completed an audited dataset, static context extraction, project-separated model experiments, a prospective three-project confirmation, source-linked explanations and a separate all-data deployment fit.

Context showed a development ranking signal and fewer observed external false alarms. Independent flaky-test recall and superiority to CANNIER remain unestablished.

Stage	Cases / projects	Observed positives
Development corpus	96,118 / 35	693 (0.721%)
Independent confirmation	2,610 / 3	0
Deployment fit after evaluation	98,728 / 38	693

Development evidence

In the bounded nested development study, revised-context Extra Trees reached average precision 0.0552 versus 0.0071 for the revised core. The project-resampling interval for the difference was [-0.0065, 0.0855], including zero. These inspected folds are development evidence.

At the top 10% ranked separately within each development project, context/core models recovered 121/693 (17.46%) and 103/693 (14.86%) observed positives. No execution-time saving was measured.

Prospective confirmation: false-alarm results

Frozen model	Flags / 2,610	Per 1,000 negatives
Extra Trees: scope-aware core	111	42.529
Extra Trees: core + context	3	1.149
Random Forest: core reference	24	9.195
Random Forest: context reference	2	0.766
Constant-negative reference	0	0.000

All confirmation cases passed 25 rounds. Recall and average precision are undefined. Models use separately frozen development-selected thresholds; different selected class weights and thresholds prevent a pure causal attribution to context. A constant-negative classifier also makes zero false alarms here.

The evaluated corpus

Model-eligible cases. Negatives mean not observed flaky after the recorded finite run budget.

Project	Cases	Observed flaky	Prevalence
airflow	3,225	300	9.30%
celery	2,332	7	0.30%
cirq	12,189	16	0.13%
conan	3,047	24	0.79%
dask	8,526	1	0.01%
djangorestframework	1,402	1	0.07%
electrum	542	0	0.00%
filelock	1,106	2	0.18%
flexget	1,330	4	0.30%
fonttools	3,448	0	0.00%
freezegun	103	15	14.56%
graphene	346	1	0.29%
hydra	1,538	18	1.17%
hypothesis	4,335	2	0.05%
ipython	801	35	4.37%
kombu	785	20	2.55%
libcloud	8,227	132	1.60%
loguru	1,359	20	1.47%
mitmproxy	1,232	2	0.16%
pillow	2,596	0	0.00%
prefect	7,046	12	0.17%
pygithub	711	0	0.00%
pyramid	2,629	1	0.04%
pytest	4,592	37	0.81%
reframe	639	9	1.41%
requests	537	0	0.00%
rich	957	5	0.52%
salt	2,674	3	0.11%
scikit_image	6,261	1	0.02%
seaborn	1,025	2	0.20%
setuptools	698	7	1.00%
sunpy	1,885	1	0.05%
tornado	1,103	0	0.00%
urllib3	2,113	4	0.19%
xonsh	4,779	11	0.23%
attrs (confirmation)	1,401	0	0.00%
tenacity (confirmation)	185	0	0.00%
werkzeug (confirmation)	1,024	0	0.00%
Total eligible / 38 projects	98,728	693	0.702%

Development source coverage: 96,118 of 99,861 candidate cases (96.25%); excluded 3,743 cases, including 22 observed positives. Original evidence remains retained. Confirmation collected 2,621 identities: 2,610 binary all-pass cases and 11 attrs skip-only identities. No confirmation binary source exclusions.

Method and fair comparisons

What was frozen before confirmation

Werkzeug, attrs and Tenacity were selected before inspecting their new outcomes. Source commits, the 30/96 feature representations, model candidate grids, project folds, selection rule, thresholds and metrics were recorded. Four final models were fitted only on the 35 development projects before opening confirmation artifacts.

Each new project passed a full baseline gate, followed by 12 original, 12 random and one reverse round on Ubuntu 24.04 / Python 3.11.9. All jobs completed within about 4.5 minutes, inside the four-hour cap. Zero-positive projects were retained.

Static features and leakage controls

The 30-feature core includes two scope-aware read indicators. The 96-feature extension adds bounded fixture, helper, lifecycle and class-state counts. Runtime outcomes, repeated-run counts, identities and seeds are label/provenance evidence only. No synthetic oversampling or artificial duplication was used.

The external feature adapter exactly reproduced 7,680 numeric values across 80 development contexts. Frozen external model predictions and metrics passed 109 checks. The existing suite ran 85 tests: 84 passed and one Linux-specific test skipped.

Published work: compatibility matters

The 2024 IEEE Access cross-project artifact uses Java metrics. Its transfer experiment trains with labels from 25%, 50% or 75% of the target project and evaluates a separate target subset. Our new-project task has no target training labels. Direct score comparisons would be invalid.

Our 100-tree Random Forest references use our Python core/context features and the same external cases. They omit sampling, PCA and target-label transfer. They are model-family adaptations, not faithful paper reproductions. No complete CANNIER hybrid baseline or matched rerun-efficiency experiment was reproduced.

Explanations and uncertainty

Nine source-linked examples decompose the Extra Trees score into an average tree-root value plus contributions along observed decision paths. Every explanation reconstructs the score. These are tree-path attributions, not SHAP or causal diagnoses. Global impurity importances have known attribution biases.

Random rounds change both order and runtime seeds; order-dependency subtypes remain provisional. Plugin-aware fixture lookup, dynamic dispatch, near-clone checks and human cause validation remain incomplete. Only three external projects were sampled, with no observed positives.

Artifacts and guide decision

A usable all-data deployment model

After independent evaluation, we fitted a separate context Extra Trees model on 98,728 eligible cases across 38 projects, with 693 observed positives. It retains the development-selected configuration and threshold 0.036106.

This deployment model does not own the earlier confirmation score. It has seen those projects during fitting and has no new unbiased test result. Its serialization was replayed on 100 cases. Risk scores are uncalibrated estimates, not confirmed flaky labels.

What is available to review

FINAL_STUDY_REPORT.md: measured findings, limitations and reproduction notes.

GUIDE_FINAL_BRIEF.md: a concise meeting brief.

external_metrics.csv / external_predictions.csv.gz: frozen model results, including undefined-metric flags.

protocol.json / model_manifest.json / artifact_provenance.json: prospective design, model hashes and raw-evidence references.

source_linked_explanations.json: nine checked attributions with source hashes.

MANUSCRIPT_DRAFT.md: an empirical-study draft, explicitly marked not submission-ready.

The decision to discuss with the guide

The completed work supports an evidence-backed empirical study of bounded Python test context. A stronger detection-improvement paper needs independent positive confirmation, a compatible reproduced baseline and human validation. A deployment fit or high accuracy on a zero-positive sample cannot supply that evidence.

Recommended follow-up: prospectively define independent positive-project collection; validate extraction and cause annotations with human reviewers; then test matched recall or matched inspection budgets. Keep this confirmation unchanged. No publication acceptance guarantee is justified.

Primary references

CANNIER (2023): doi:10.1007/s10664-023-10307-w
Cross-project flaky-test prediction (2024): doi:10.1109/ACCESS.2024.3458184
Static Test Flakiness Prediction (2022): doi:10.1007/s10664-022-10227-1

Workflow: github.com/Nandinipatil1410/FlakeXplain/actions/runs/38429430361
Authors artifact inspected at commit 8fe244f03122706763484dea7b4d98b09fefb923.

Reporting clarification: the preserved evaluation metadata calls attrs' 1,412 retained identities "candidate_binary_cases"; 11 are skip-only. The corrected binary total is 1,401. Scores and metrics are unaffected; see metadata_clarification.json.